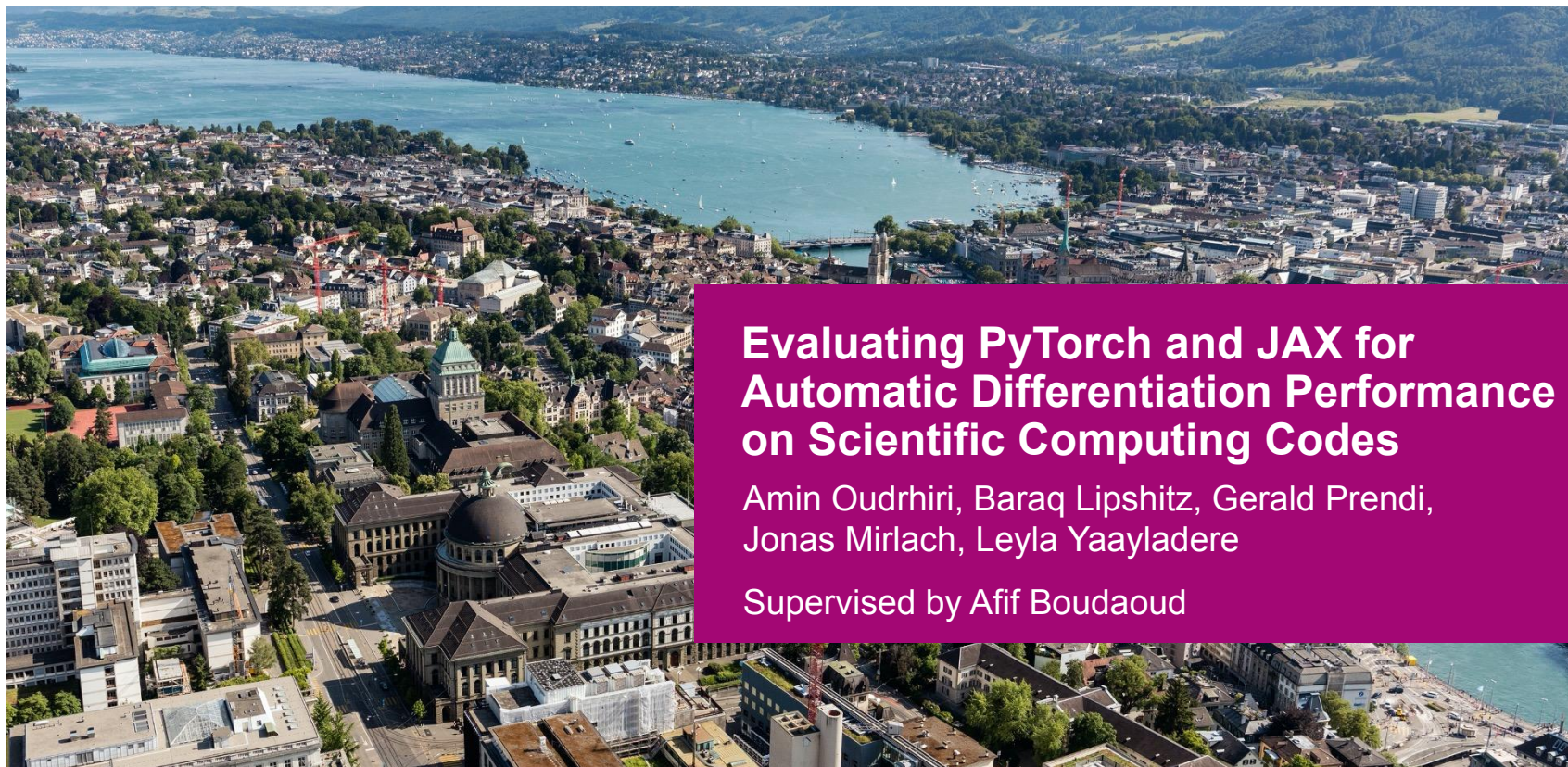




Evaluating PyTorch and JAX for Automatic Differentiation Performance on Scientific Computing Codes

Amin Oudrhiri, Baraq Lipshitz, Gerald Prendi,
Jonas Mirlach, Leyla Yaayladere

Supervised by Afif Boudaoud

The goal of our project is to benchmark Automatic Differentiation in NPBench



Automatic Differentiation (AD) computes gradients in ML and scientific applications.



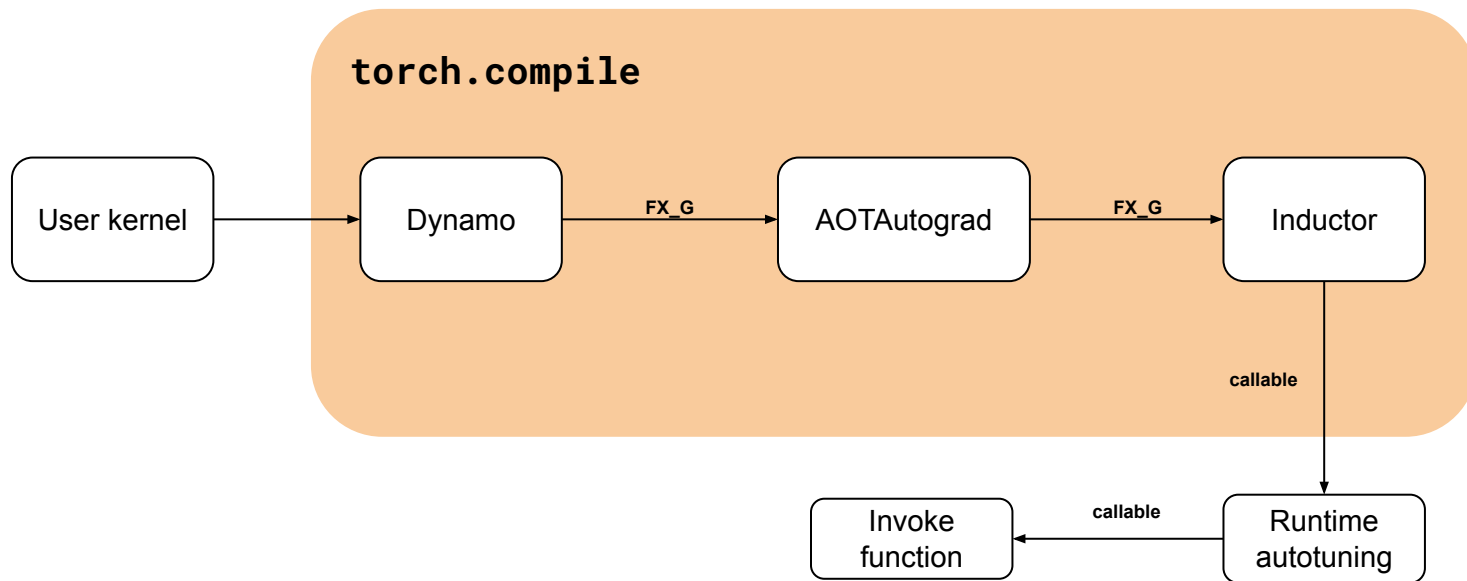
Problem: Existing ML frameworks (e.g., PyTorch, JAX) are not optimized for complex scientific code patterns.



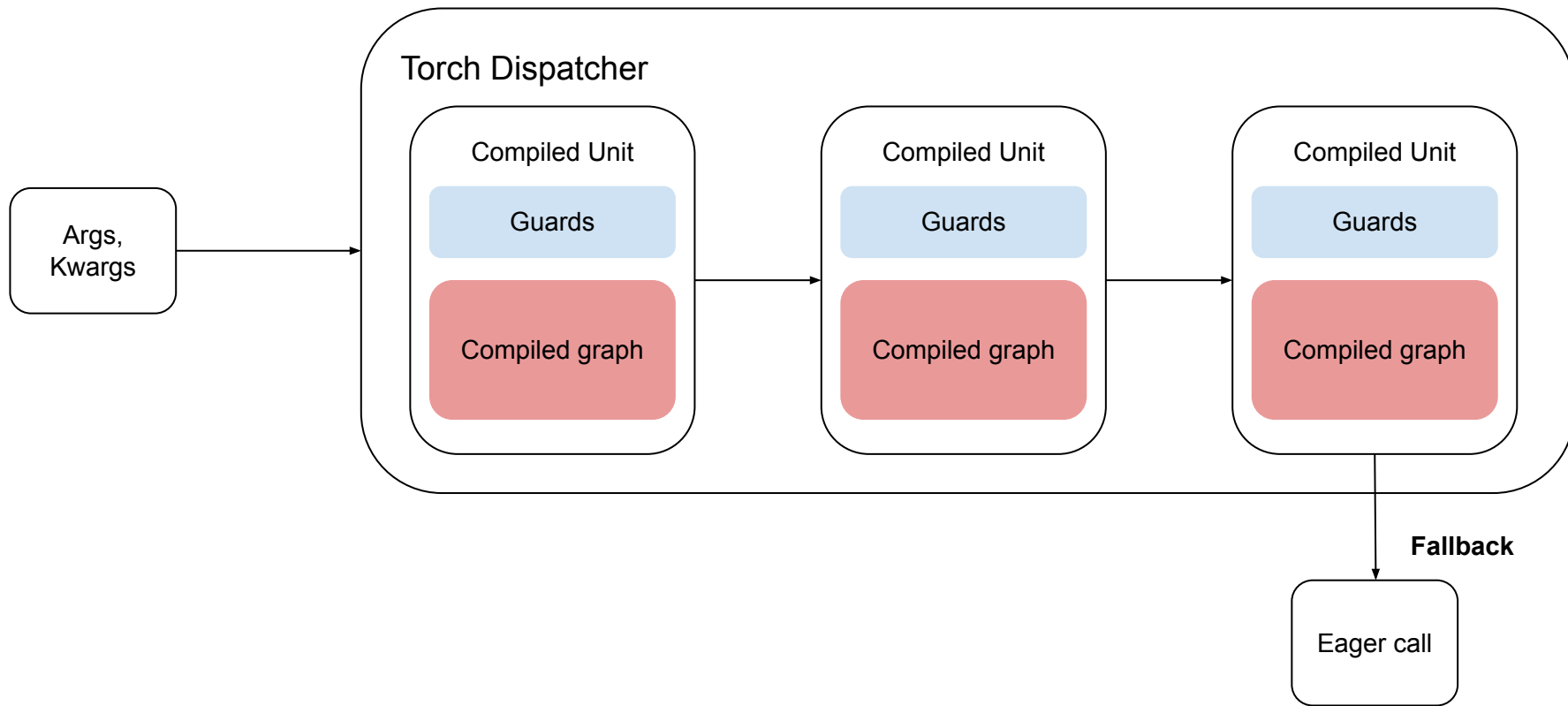
Goal: Evaluate and compare the reverse-mode AD performance of PyTorch and JAX on NPBench scientific kernels.

Background: Framework Compilation Differences

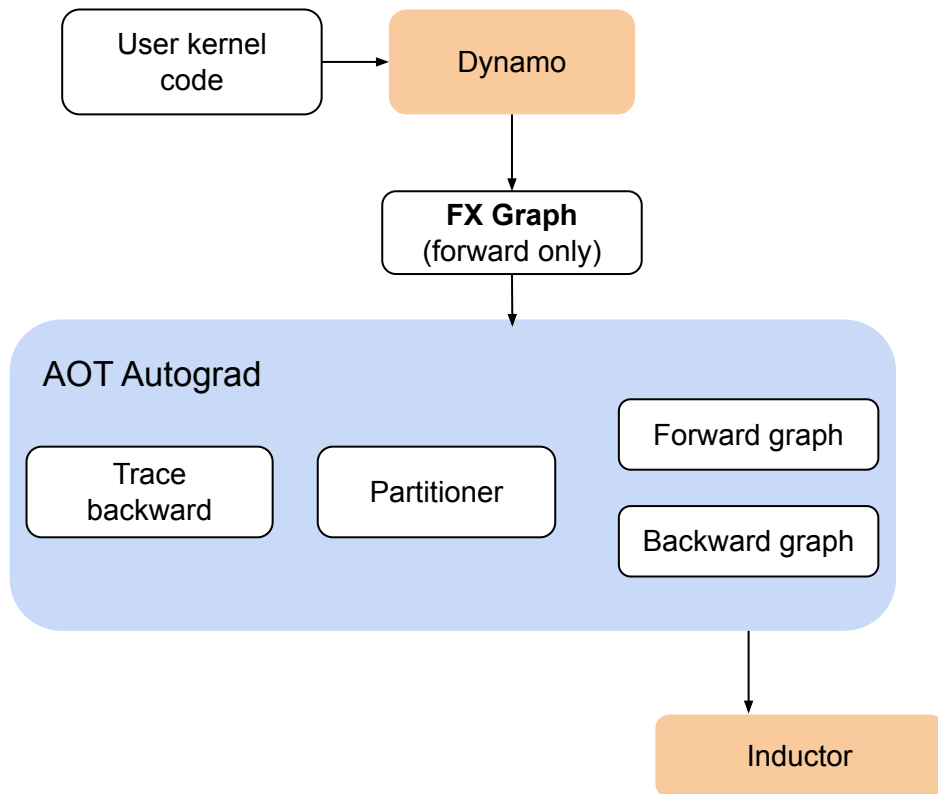
Overview of the PyTorch JIT compilation pipeline



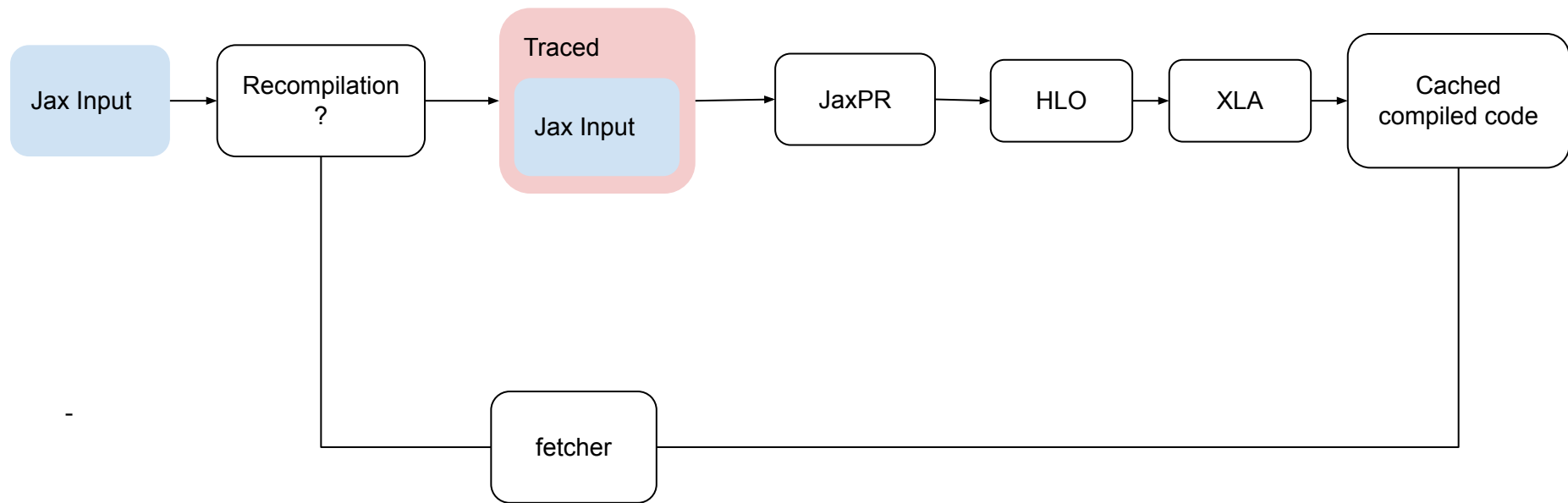
Deciding which compiled function to call



AOTAutograd's compilation pipeline



Understanding JAX's `jit` compilation pipeline



How tracing and `jax.grad` interact

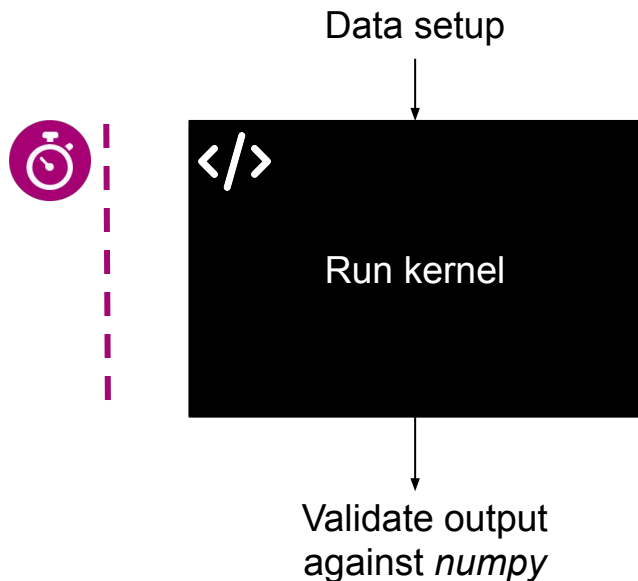
- Takes an ahead of time approach
- Relies on a set of primitives to explicitly declare the dynamic behaviour: `lax.cond`, `lax.fori_loop`
- Notion of **static** arguments



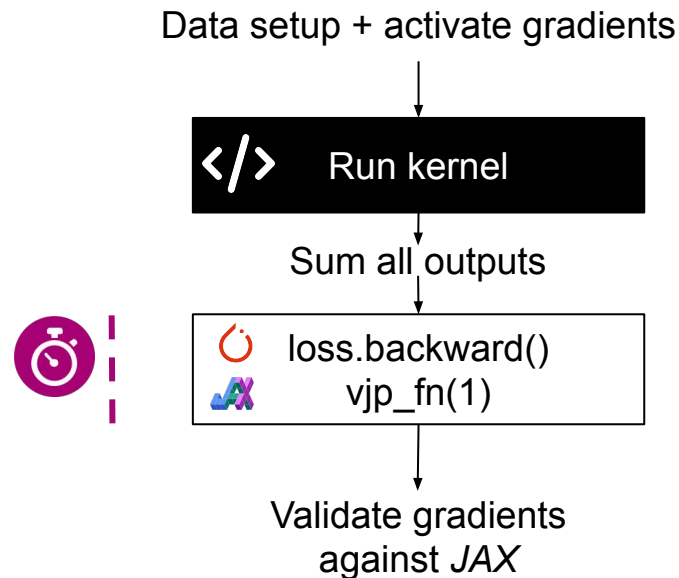
Our Experimental Setup

We only measure the plain computation time of the backward pass

Forward kernel run



Backward (Reverse-mode AD)



Ensuring a fair JAX and PyTorch comparison



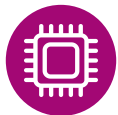
PyTorch implementations aim to be similar to given JAX implementations, not to NumPy



One main difference in code: PyTorch in-place backprop requires `.clone()`



5 repeats with warmup, data size selected so it lands >100ms



Xeon E5-2680 v4 CPU and NVIDIA GeForce RTX 5060 Ti GPU

Pattern #0:
Linear Algebra



PATTERN DEFINITION

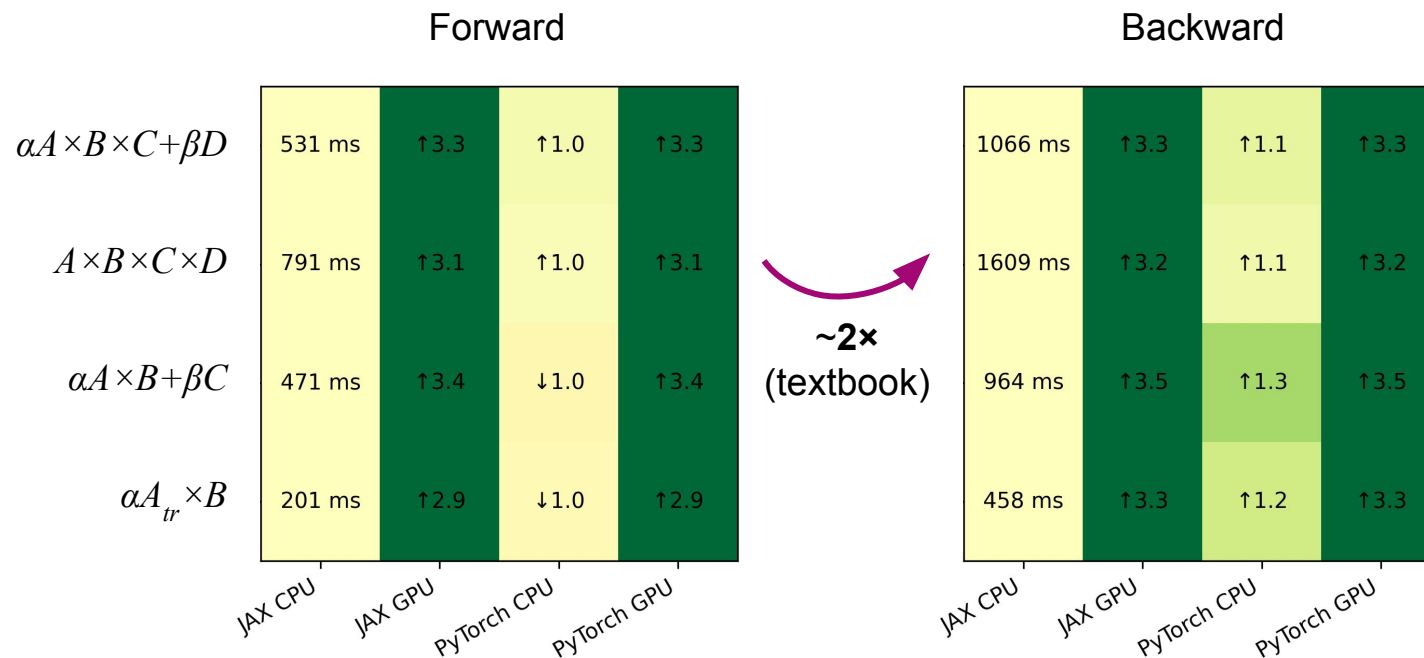
Matrix Multiplications, Matrix-Vector Products, no explicit loops

Example kernel:

```
def k2mm(alpha, beta, A, B, C, D):  
    return alpha * A @ B @ C + beta * D
```

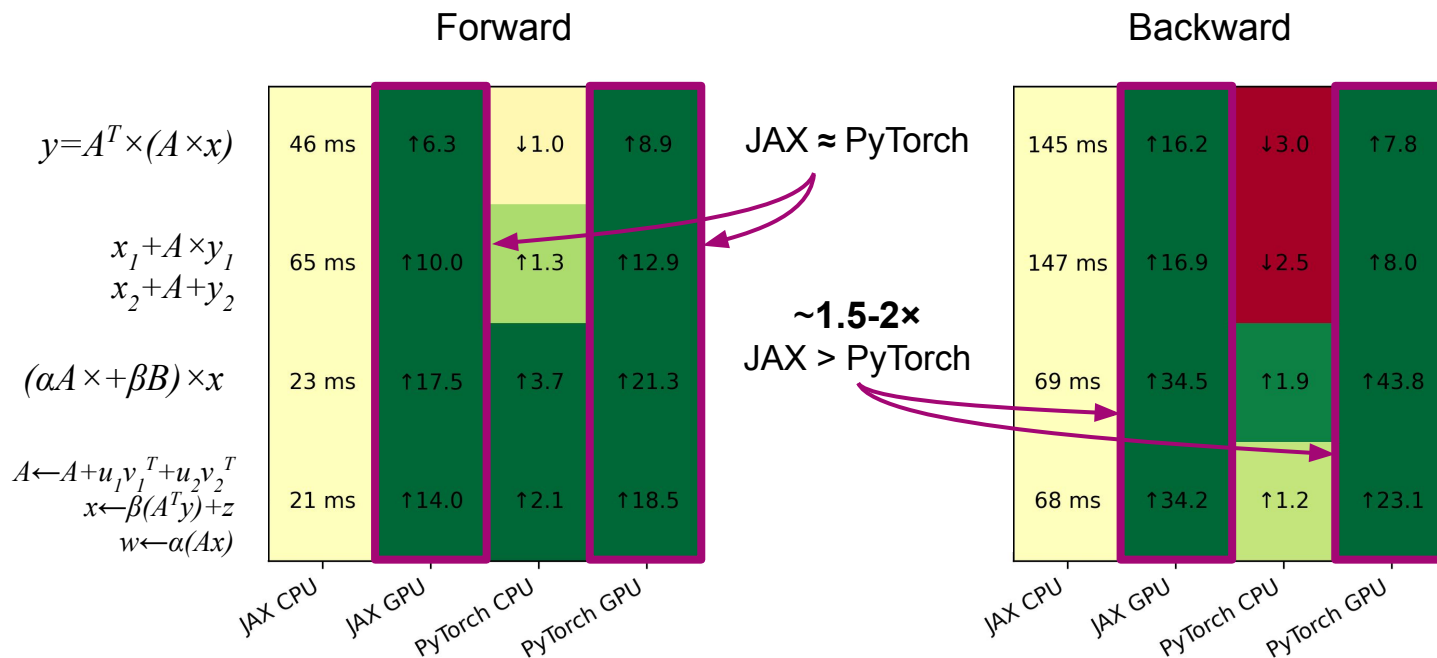


RESULTS I





RESULTS II



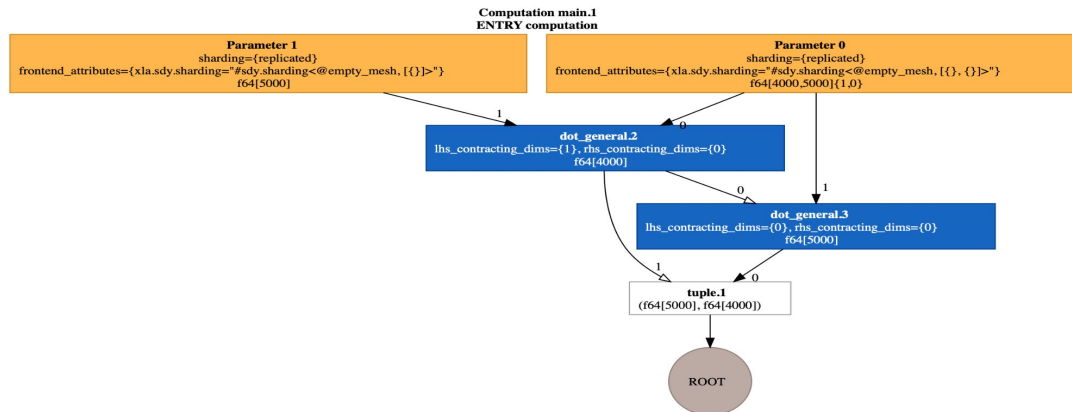
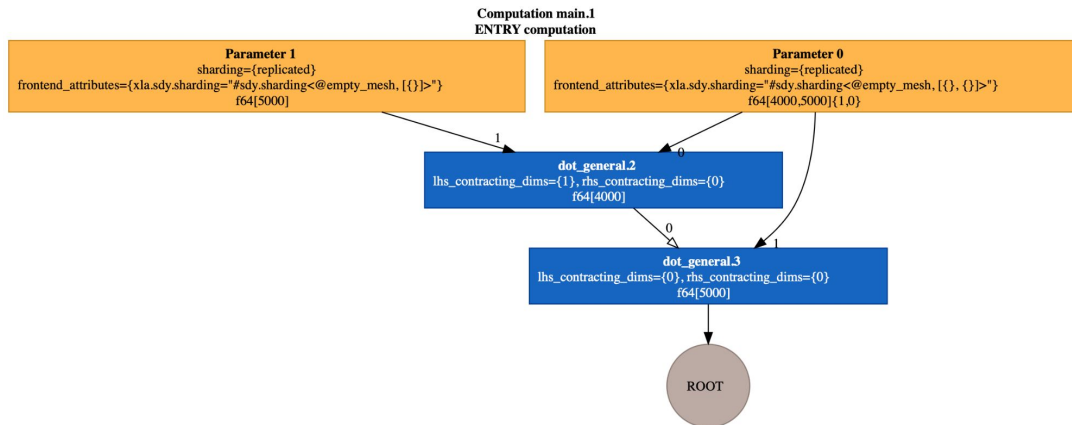
Atax forward

#0 Linear Algebra

Graph on the right, top to bottom:
Atax Forward (no grad computation)
Atax Forward for Gradient Computation

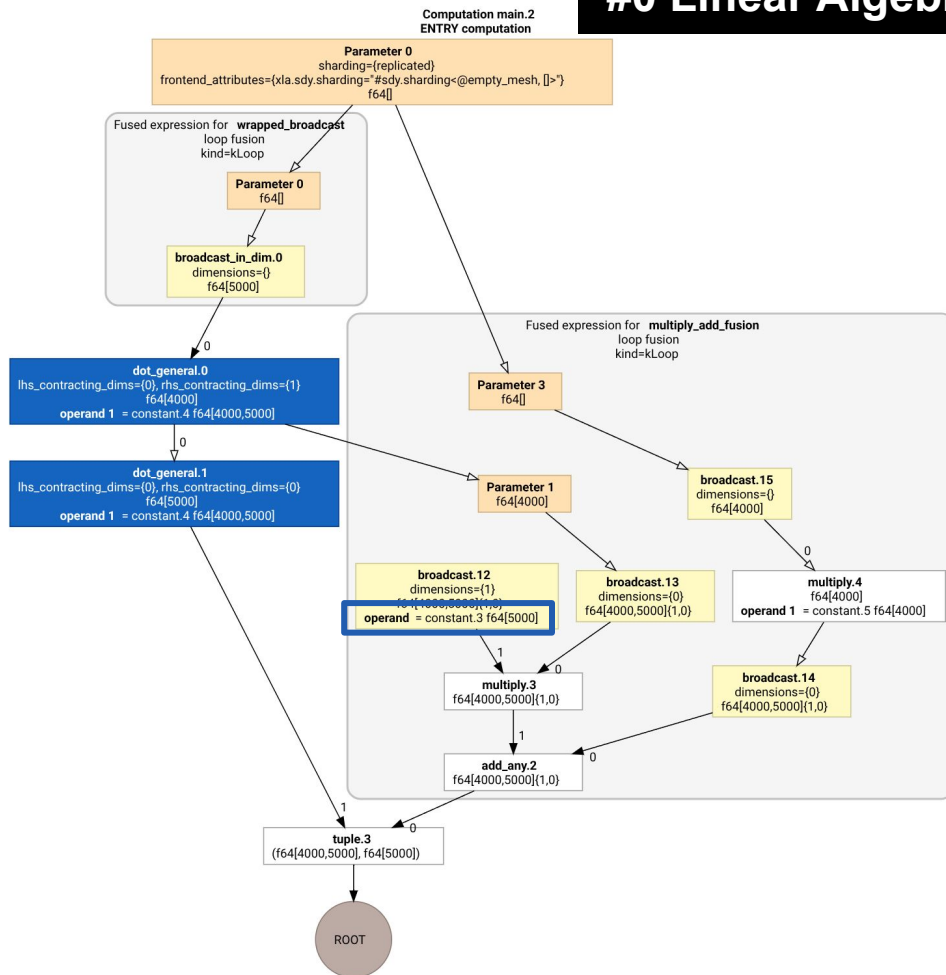
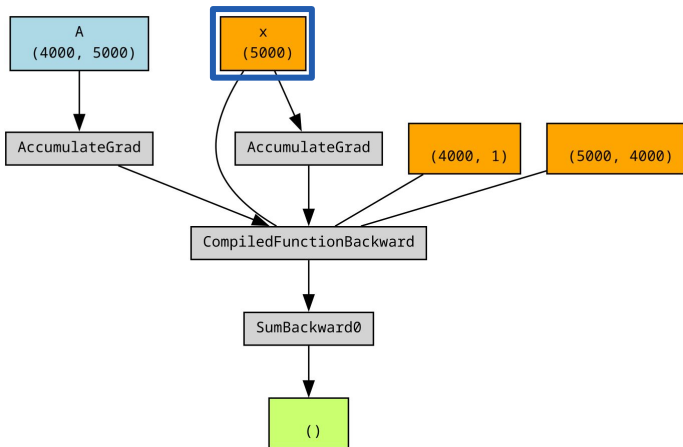
Logic: Executes standard matrix multiplication but splits the graph to return a **tuple**: the final result **y** plus the **intermediate residual A @ x**.

Graph Signature: Linear algebra nodes (**dot_general**) ending in a **tuple** output to preserve state for the backward pass.



PyTorch vs JAX backward

#0 Linear Algebra





PATTERN DEFINITION

Matrix Multiplications, Matrix-Vector Products, no explicit loops



INSIGHTS

1. **Compute-bound (BLAS/GEMM) ops perform same** — for GPU, JAX and PyTorch hit the same cuBLAS kernels
2. **CPU performance can vary (up to 2×)** — performance is driven by which BLAS library each framework link (e.g., MKL/OpenBLAS)
3. **More overhead-heavy ops tend to favor JAX** — additional overhead in PyTorch's eager execution via `.backward()`

PyTorch's `torch.compile` unrolls loops

anijain2305 on Aug 8, 2024

Contributor ...

TorchDynamo still unrolls the loop aggressively. There is no easy workaround here. If it makes sense in your codebase, instead of applying `torch.compile` at the very top, you could lift the loop body into a separate function and then apply `torch.compile` manually on that lifted function. This might be not very user-friendly.



JAX's `fori_loop` enables feasible full-function compilation,
for PyTorch only loop body compilation is feasible



```
@jax.jit
def go_fast(a: jax.Array):
    trace = 0.0
    def body_fn(i, trace):
        trace += jnp.tanh(a[i, i])
        return trace
    trace = jax.lax.fori_loop(0, a.shape[0], body_fn, trace)
    return a + trace
```



```
def go_fast(a: torch.Tensor) -> torch.Tensor:
    @torch.compile
    def body_fn(trace, diag):
        return trace + torch.tanh(diag)

    trace = torch.tensor([0.0], dtype=a.dtype, device=a.device)
    for i in range(a.size(0)):
        trace = body_fn(trace, a[i, i])
    return a + trace
```

Pattern #1:
Simple Loop Bodies



PATTERN DEFINITION

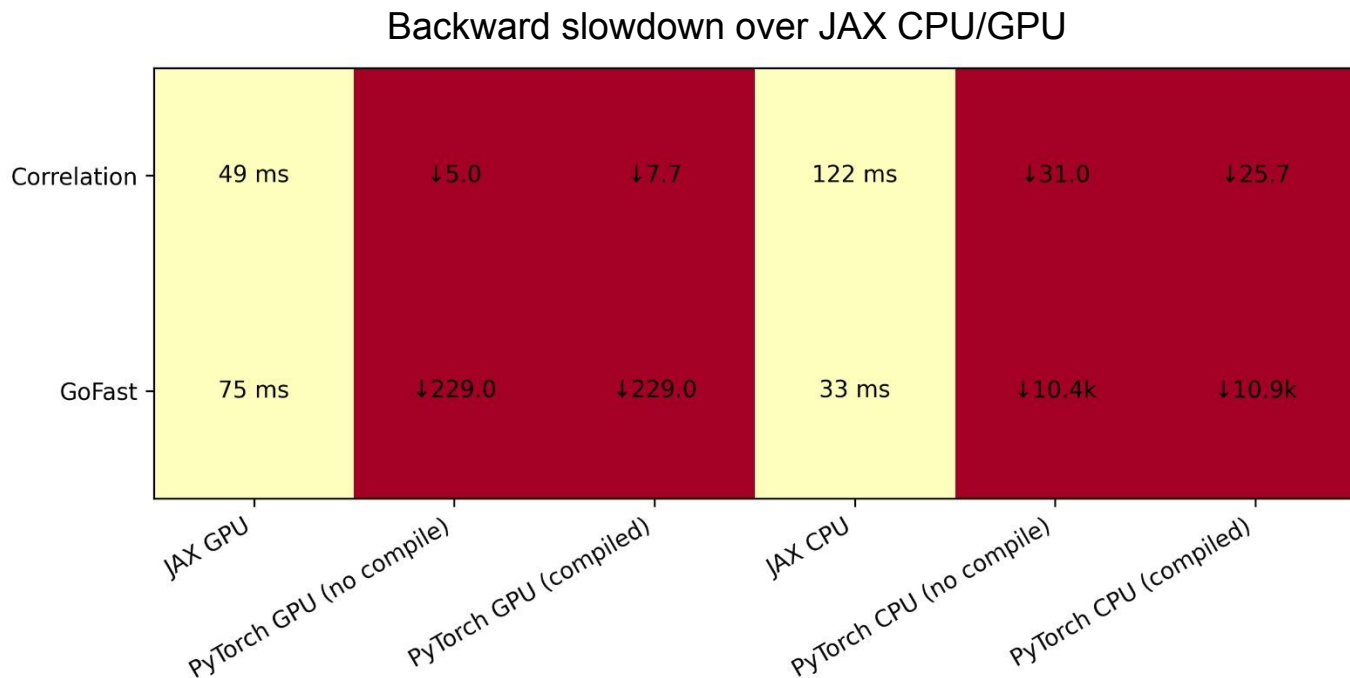
Loops with a minimal body (few operations), embarrassingly parallelizable across iterations

Example kernel:

```
def go_fast(a):  
    trace = 0.0  
    for i in range(a.shape[0]):  
        trace += tanh(a[i, i])  
    return a + trace
```



RESULTS I: JAX outperforms PyTorch. Simple loop body compilation yields no benefits

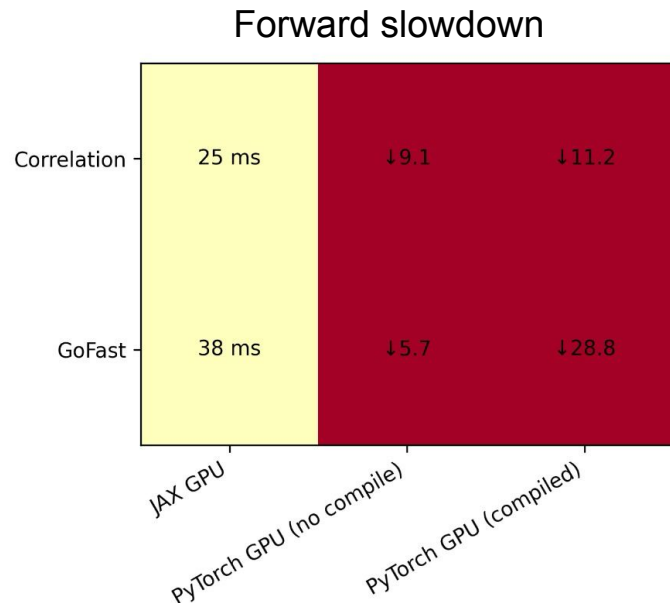
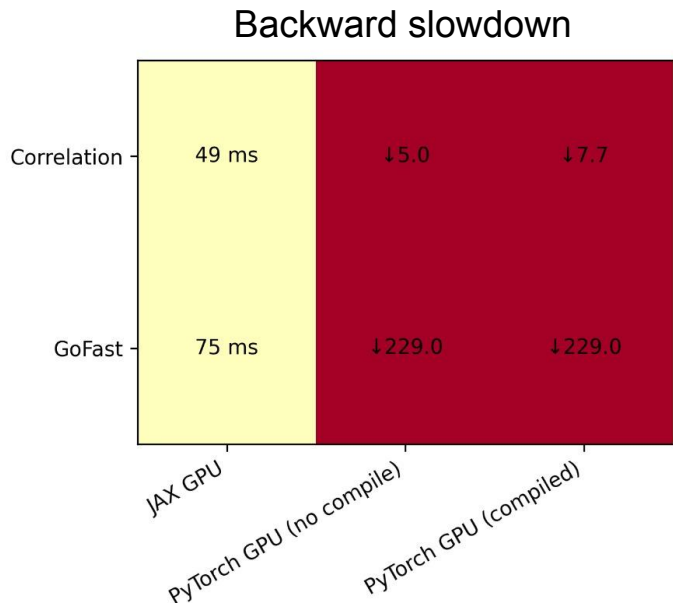


*CPU excluded for simplicity, similar results.

**Larger data size impossible due to student-cluster constraints.



RESULTS II: Torch underperformance is worse in the backward case



*CPU excluded for simplicity, similar results.

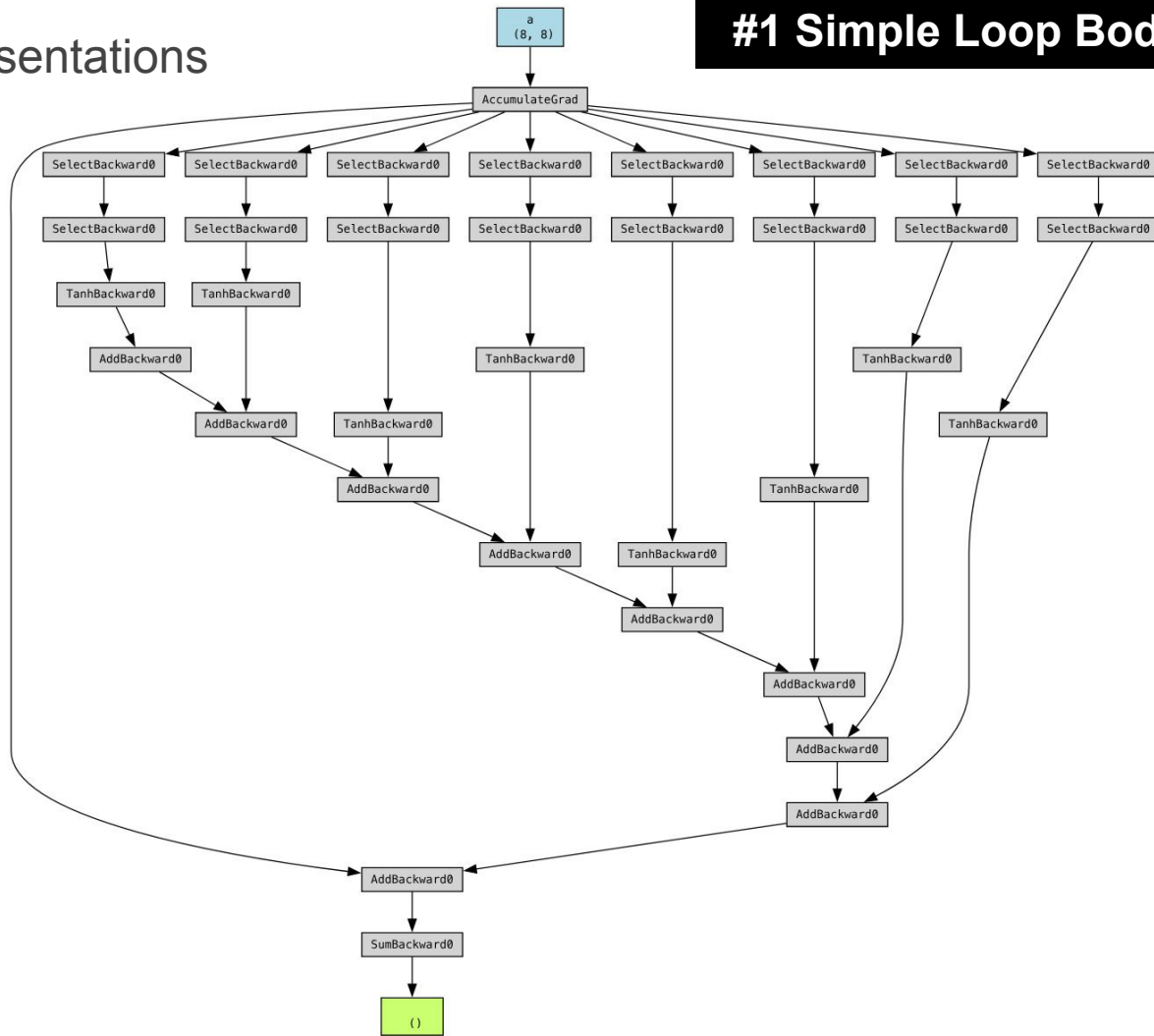
**Larger data size impossible due to student-cluster constraints.

PyTorch Intermediate Representations

go_fast

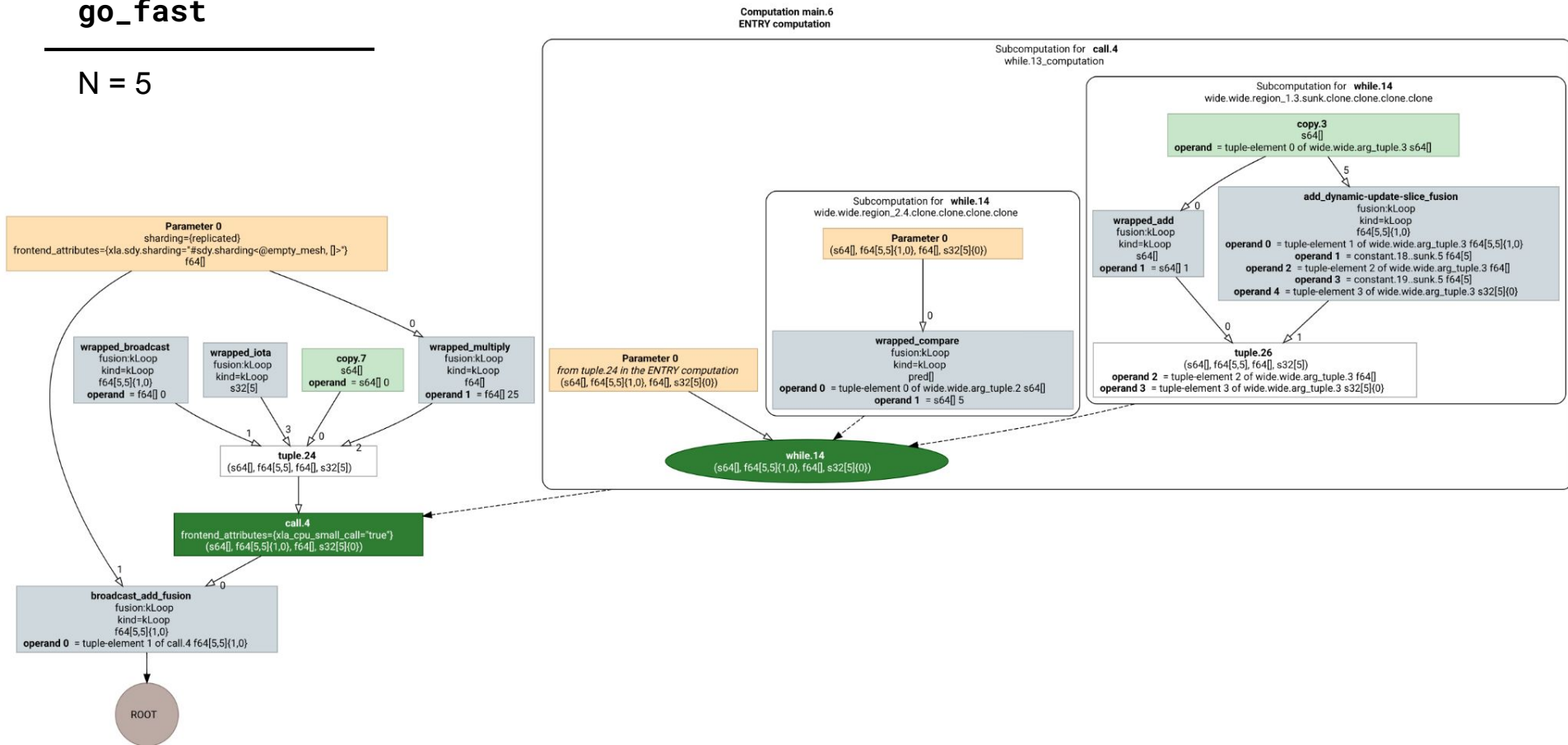
N = 5

#1 Simple Loop Bodies



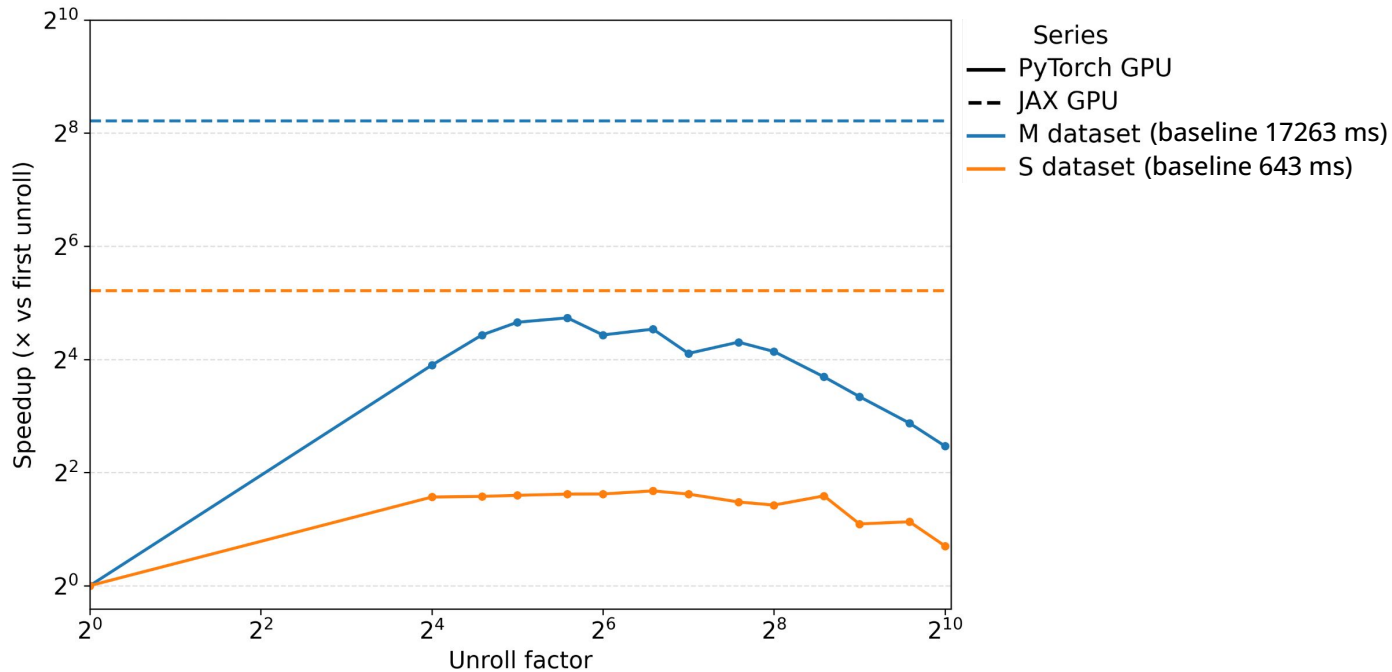
#1 Simple Loop Bodies

N = 5



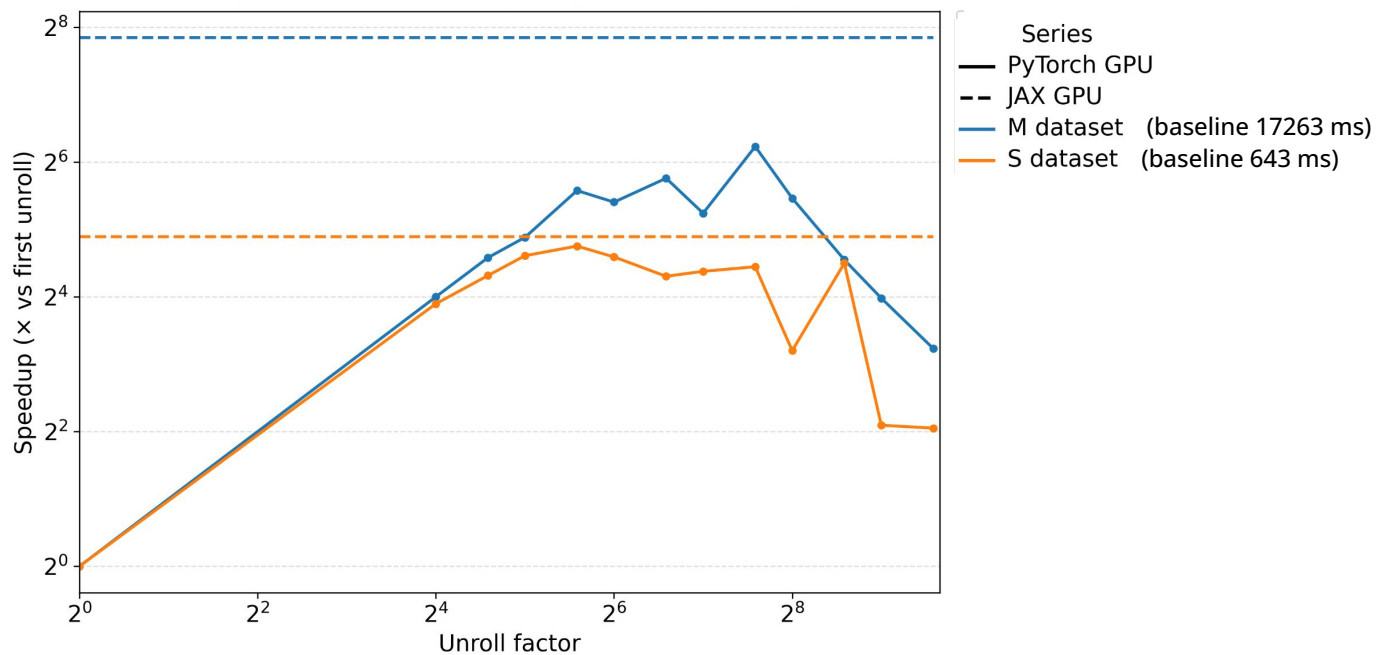


RESULTS III: Unrolling PyTorch increases speed





RESULTS IV: Unrolling and compiling PyTorch approaches JAX performance





PATTERN DEFINITION

Loops with a minimal body (few operations), embarrassingly parallelizable



INSIGHTS

1. JAX handles the control flow much better in both autodiff graph construction and kernel launches.
2. PyTorch loop body compilation is not beneficial when loop body is trivial.
3. Unrolling a simple inner loop in compiled PyTorch function gives a large speedup over a naive implementation.

Pattern #2:
Static Loop Bodies



PATTERN DEFINITION

Sequential and dependent loops, loop body is highly parallelizable

Example kernel:

```
def jacobi1D(TSTEPS, A, B):  
    for t in range(1, TSTEPS):  
        B[1:-1] = 0.33333 * (A[:-2] + A[1:-1] + A[2:])  
        A[1:-1] = 0.33333 * (B[:-2] + B[1:-1] + B[2:])  
    return A, B
```

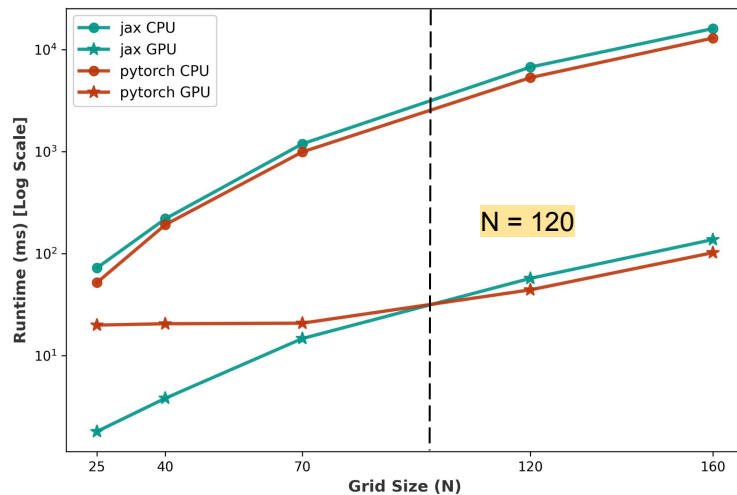
This pattern is valid for all **Iterative Stencil Computations** such as `jacobi_1d`, `jacobi_2d`, `fdtd_2d` and `heat_3d`.



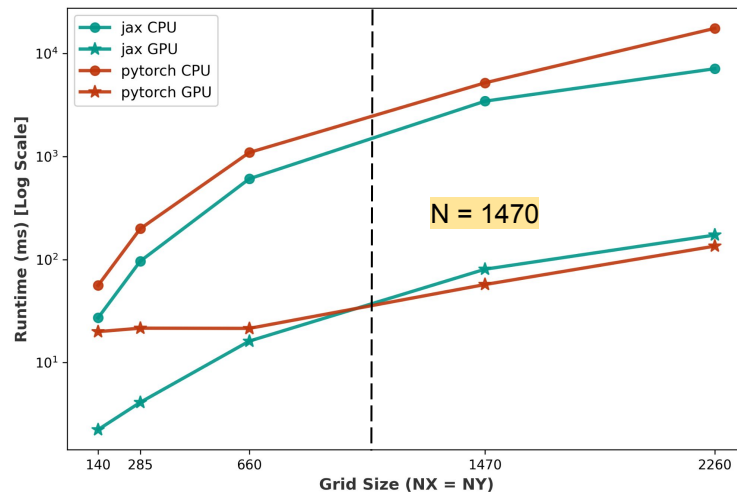
RESULTS I : PyTorch Matches Jax at High Computational Loads

Ablation Study for Grid Size (N), fixed T_STEPS = 100

heat_3d | Backward Pass



fdtd_2d | Backward Pass



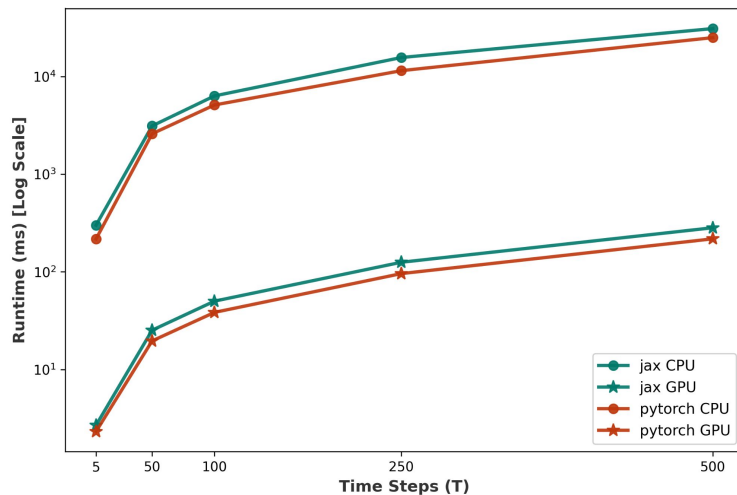
*N values are selected to have similar computational load for *Heat_3D* $\sim 15 \cdot T \cdot N^3 \approx$ *FDTD_2D* $\sim 12T \cdot N^2$



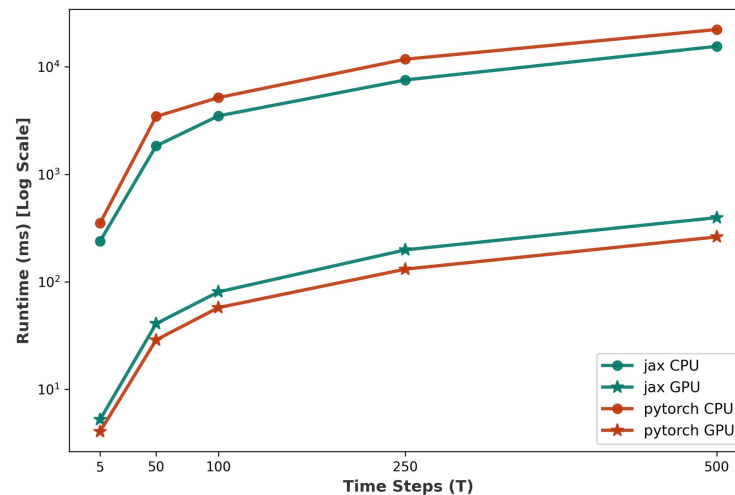
RESULTS II : Performance Parity Persists Across All Time Steps for Backward

Ablation Study for T_STEPS, fixed Grid Size (N)

heat_3d | Backward Pass

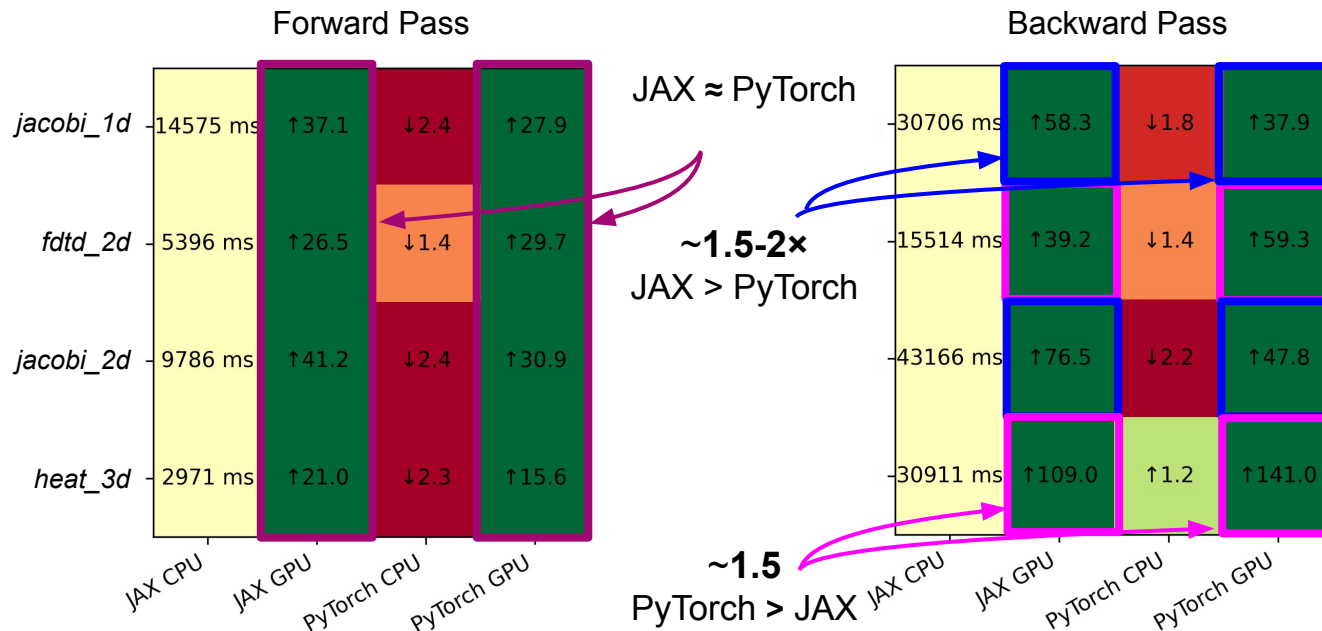


fdtd_2d | Backward Pass



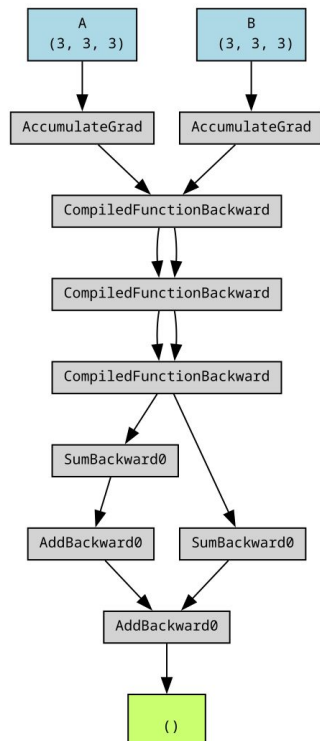


RESULTS III : More Kernels for This Pattern

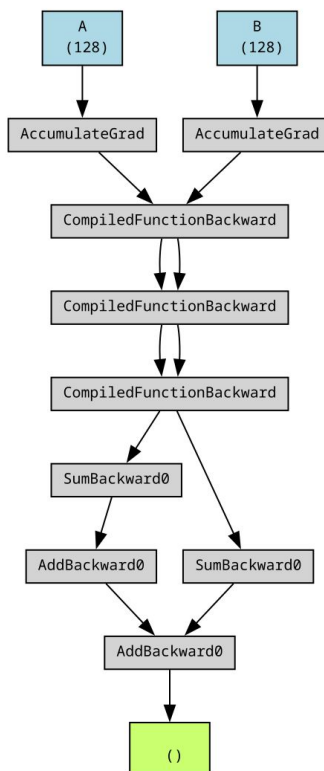


*N values are selected to have similar computational load, fixed T_STEPS = 500.

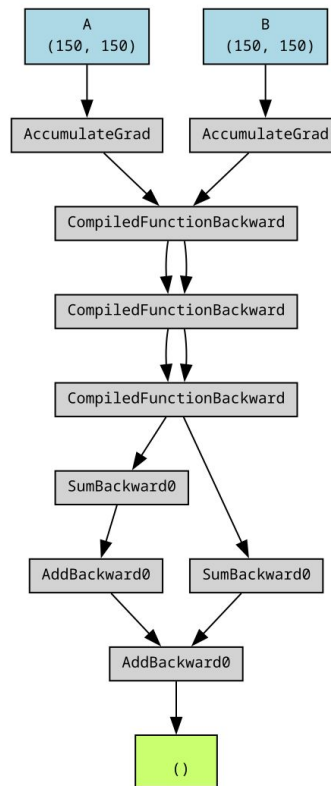
heat3d



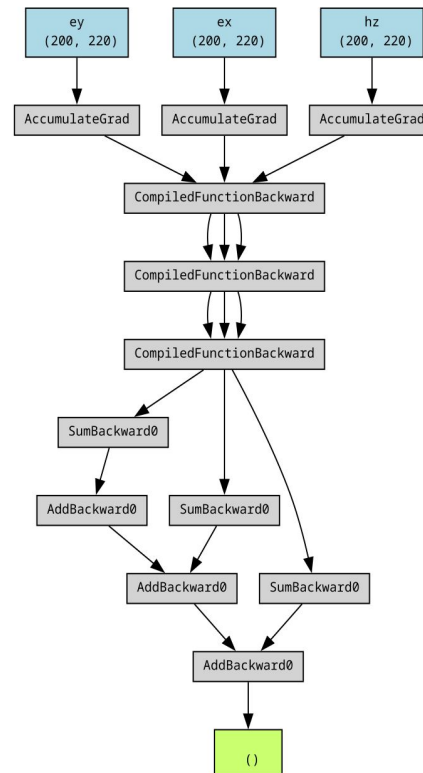
jacobi_1d



jacobi_2d



fdtd_2d





- **Direction:** Iterates 0 to T.
- **Operation:** `dynamic_update_slice` (Scatter/Write). Updates cell values based on neighbors.

- **Direction:** Iterates T to 0 (Reverse).
- **Operation:** `dynamic_slice` (Gather/Read) + `pad_add_fusion` (Accumulate).
- **Logic:** Gathers error signals from all 6 neighbors, scales them, and sums them up.



PATTERN DEFINITION

Sequential and dependent loops, loop body is highly parallelizable



INSIGHTS

1. **Confirms Pattern #0 (Compute-Bound):** When the GPU is fully occupied by computation of loop body, JAX and PyTorch perform similarly, consistent with BLAS results.
2. **Confirms Pattern #1 (Dispatch Overhead):** For smaller grids, PyTorch lags behind due to CPU kernel launch overhead that isn't hidden by lightweight loop bodies.
3. **Saturation Threshold:** Framework choice and iteration count becomes negligible beyond a specific problem size (N) where GPU saturation makes compute cost dominate runtime, hiding PyTorch's dispatch overhead.

Pattern #3:
Dynamic Loop Bodies



PATTERN DEFINITION

Loop body depends on index and iteration count depends on data size

Example kernel:

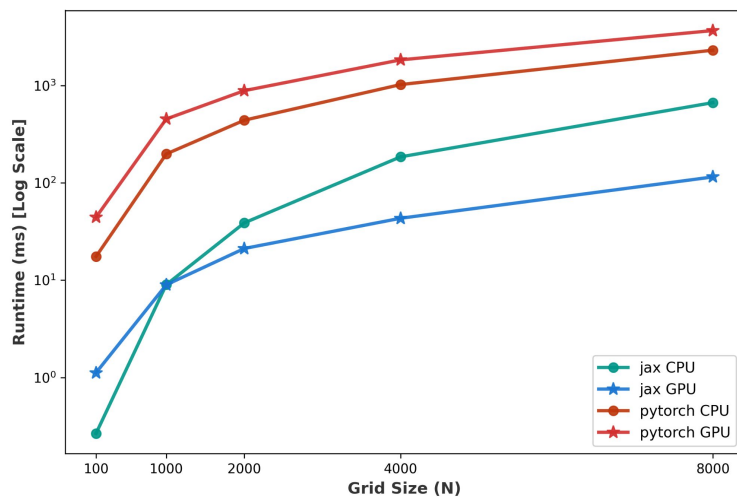
```
def durbin(r):  
    ...  
    for k in range(1, r.shape[0]):  
        ...  
        y[k] = -(r[k] + dot(flip(r[:k]), y[:k])) / beta  
    return y
```



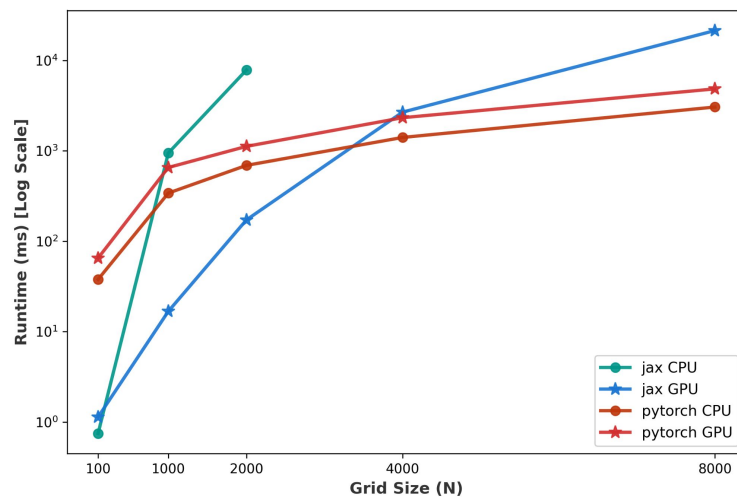
RESULTS I : Dispatch Overhead Amortization: The Impact of Scale

Ablation Study for Grid Size (N) which is also iteration count

durbin | Backward Pass



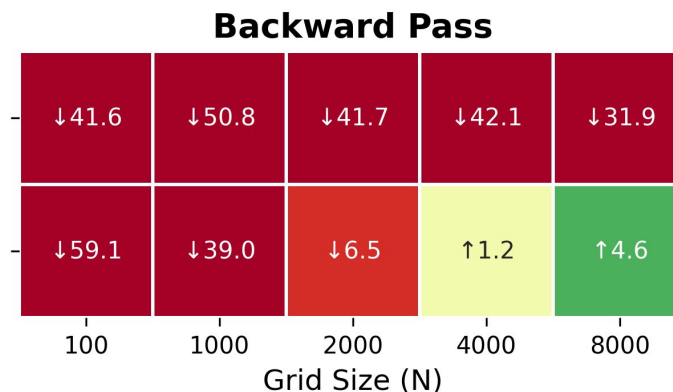
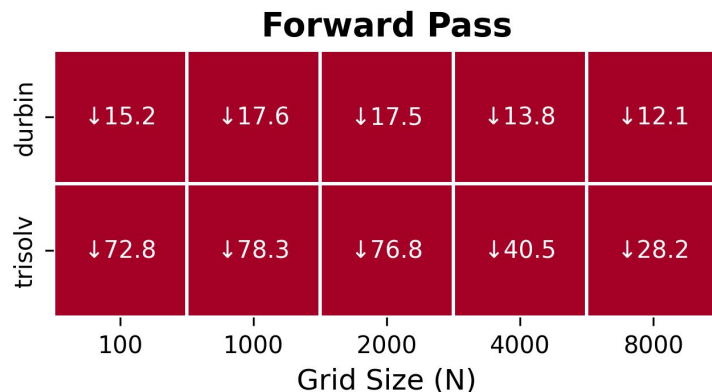
trisolv | Backward Pass





RESULTS II : Performance Gap Closes with Higher N, Faster for Complex Loop Body

"PyTorch vs. JAX GPU: Relative Factor" (Baseline: JAX GPU Runtime)

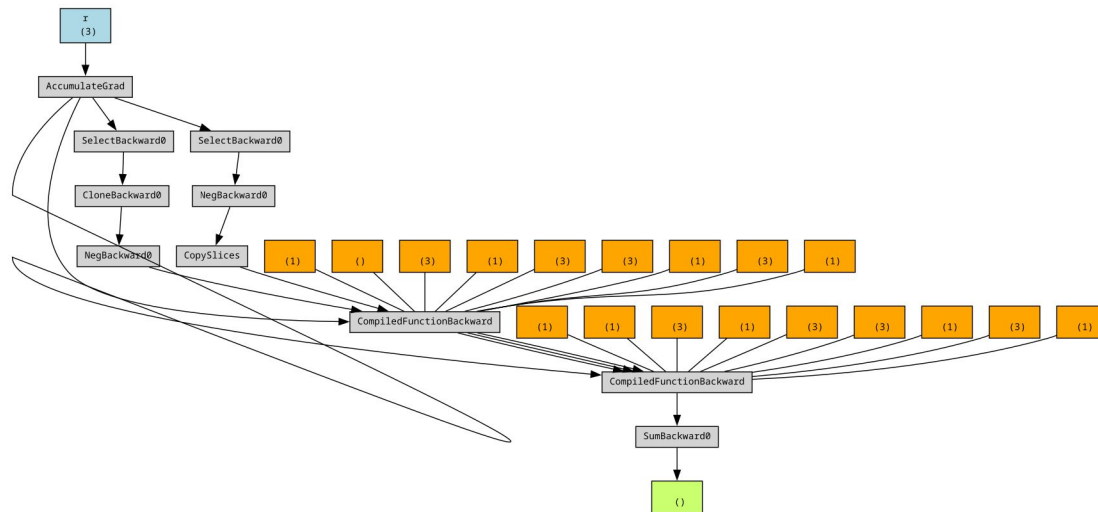
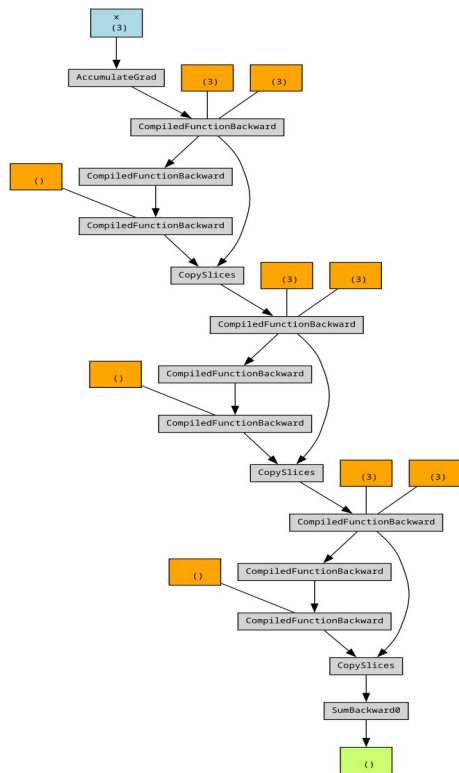


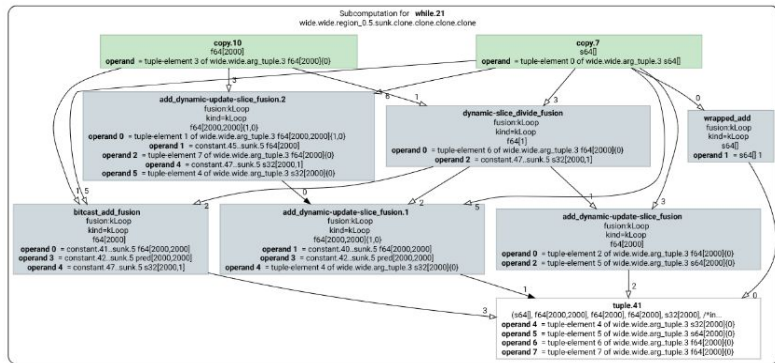
* **trisolv** operates on 2D data $N \times N$, whereas **durbin** operates on 1D data N , therefore the performance gap closes significantly faster for **trisolv**.

Backward computation

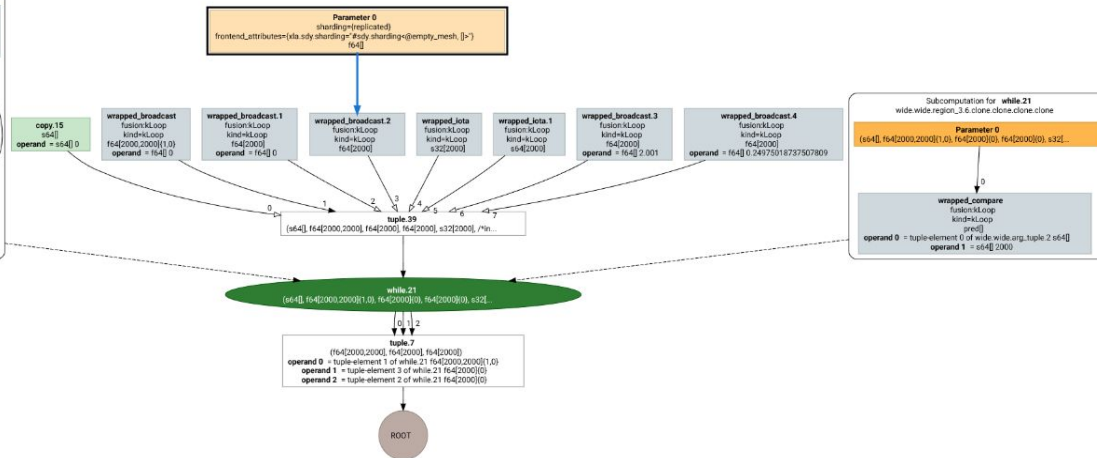
Trisolv on the left

Durbin on the right





Computation main.8
ENTRY computation



Logic: Performs **Backward Substitution** (solving the adjoint system) to propagate error signals. It iterates through the matrix to compute how changes in **L** and **b** affect the final **x**.

Graph Signature: A sequential **while** loop dominated by **add_dynamic_update_slice** operations, which scatter-accumulate error signals back into the gradient tensors for **L** and **b** iteration-by-iteration.



PATTERN DEFINITION

Loop body depends on index and iteration count depends on data size

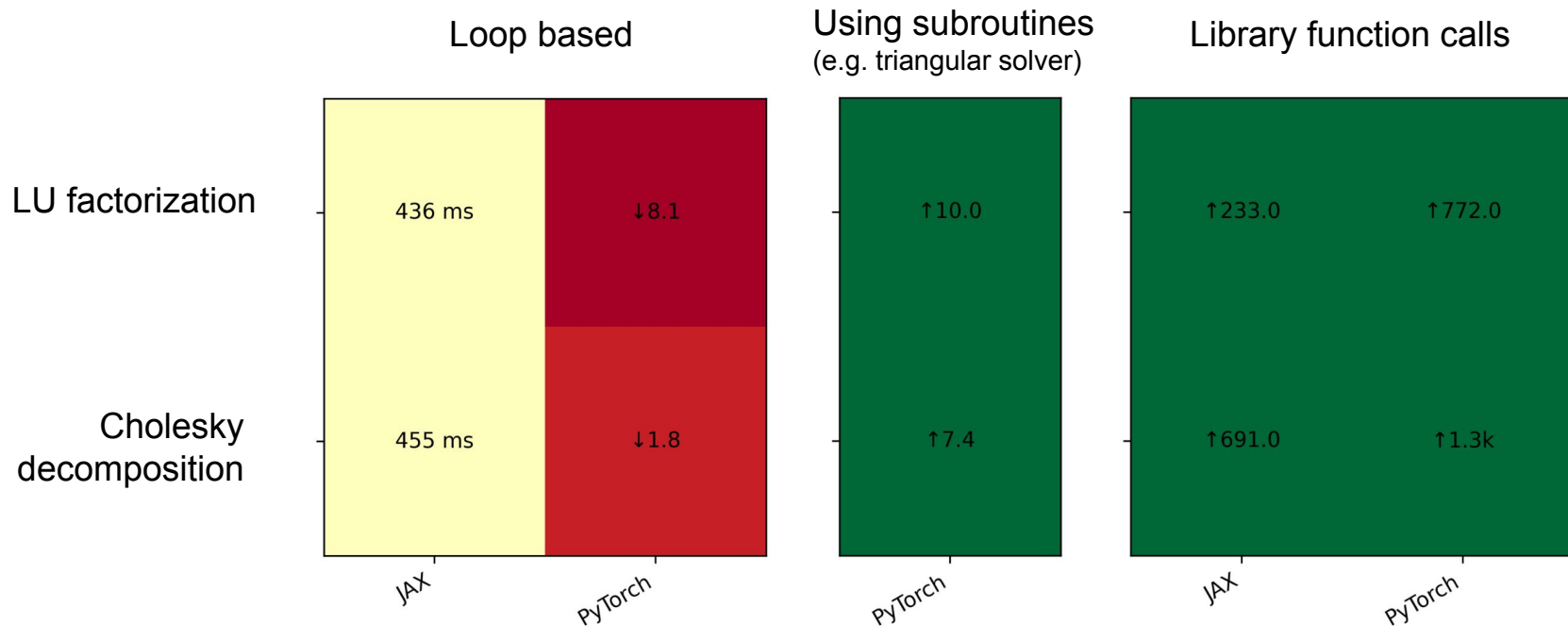


INSIGHTS

1. **Variable Workload:** Computation is index-dependent, with early iterations lighter than later ones.
2. **Saturation Threshold:** Identifying saturation is complex due to variable workload and iteration count scaling with input size (N). High arithmetic intensity bodies ($O(N^2)$) eventually mask dispatch overhead, while linear bodies ($O(N)$) often don't offset overhead from lighter early iterations.
3. **Convergence:** PyTorch matches JAX in `trisolv` (2D data) as computation dominates overhead. For `durbin` (1D data), overhead remains significant, so PyTorch catches up slowly and may never fully close the gap.

Further Loop Kernels (Nested Loops)

Highly optimized subroutines and function-calls can massively speed-up the runtime and lead to superior PyTorch performance



Conclusion

JAX AD is better suited for scientific computations with long non-parallelizable loops over PyTorch AD

- **For BLAS dominated operations:** JAX and PyTorch perform similarly both in forward and backward. **PyTorch tends to be slightly more performant.**
- **For loop heavy calculations:** **JAX outperforms PyTorch** in forward and backward AD, since PyTorch does not have an equivalent to JAX's `fori_loop` which makes efficient loop compilations possible. Tricks like loop-body compilations or batch-unrolling can make PyTorch achieve similar results to JAX but require additional engineering efforts.
- Overall, the performance of a kernel depends on the balance of control flow overhead to BLAS computations.